告别 AI Demo 页面：构建支持淘内货品运营、可C端投放的会场 Agent Harness

原创 营销&交易技术 大淘宝技术 2026年10月19日 00:24 浙江 1人



本文介绍了方舟 AI 会场 Agent Harness 如何突破传统 AI 仅生成静态 Demo 页面的局限，构建出支持淘内真实货品运营且可面向 C 端投放的生产级会场系统。该 Agent 通过策划、视觉设计、数据装配与页面构建的完整链路，将用户的一句话需求转化为承载视觉规范、商品供给与渲染协议的 HTML 产物。技术上重点构建了会场域认知、业务 Agent 工作区及自迭代环境，借助 Skill 体系注入淘系设计规范与业务规则，利用 SubAgent 实现上下文隔离与并行提效，并配合多层质检与全链路观测机制保障交付质量，最终实现“想法即生成，生成即可投放”的业务目标。



在当下，让 AI “一句话生成页面”实在已经不稀奇了，直接打开千问办公 qoder 等产品 了了两句提示词，ai 就能生成**一堆好看的 html 页面**。

但这些页面真的能上线吗？很难。

能投放到手淘端内以及端外吗？不太可能，一堆 ai 直接生成的 html 无法支撑淘内的页面渲染体系，性能、埋点、商品数据动态渲染等通通不符合要求；

能进行淘内的供给挖掘和真实的商品运营吗？不太行，ai 会胡编乱造一些商品数据，尽管把商品数据给到 ai，那线上页面的商品如何个性化推荐？有的想要白底图有的想要场景图 怎么办？有些想要定坑有些想要补全利益点 怎么办？

生成的页面视觉样式能符合手淘的设计规范吗？设计不是一个简单的活，如果不加以排版、材质、情绪等设计语言的约束，尽管AI很强大也无法生成业务心中想要的。

让生成的页面“**想法即生成，生成即可投放**”，是方舟 AI 会场希望达到的目标，下面是截取的一些会场，目前淘内已经有很多页面在投放上线中。

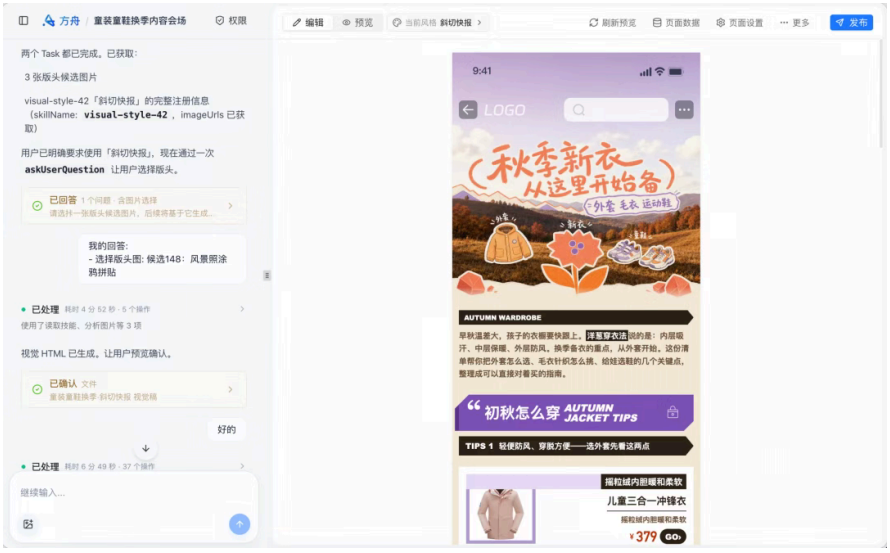


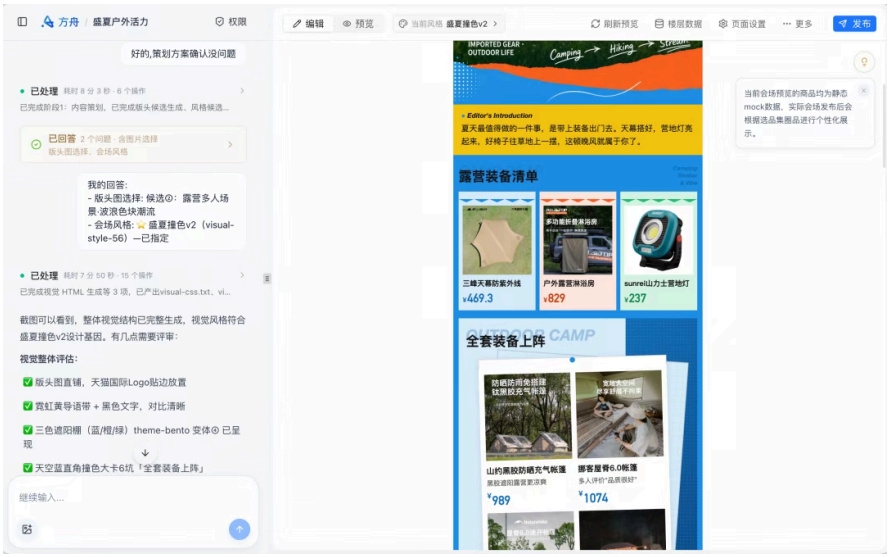
方舟会场 Agent 的生成，我们期望至少有以下几个特点：

1. 内容实时策划：基于人群/场景/热点的互联网内容深度挖掘以及策划。
2. 淘天设计水准：沉淀淘系会场设计经验，生成高颜值的精修会场
3. 真实货品运营：可以在会场中对淘内商品进行圈选，以及进行投放
4. C端线上投放：生成符合生产环境要求的会场，可以投放淘内或端外。



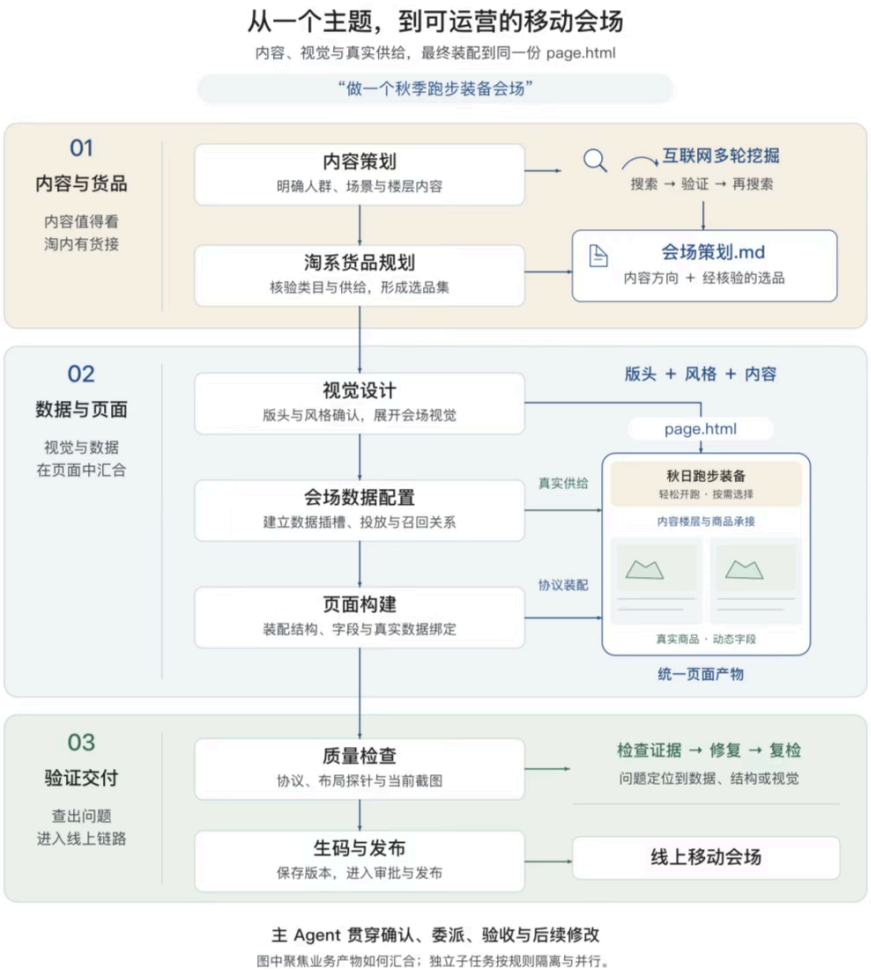
下面是 Agent 产品的一些界面：





02 整体架构

业务流程：从一句话到线上会场



1. **内容与供给产出**：策划 Skill 多轮挖掘人群/场景切口，核验类目与供给，产出《会场策划.md》与选品集 ID，交用户确认。
2. **设计与数据分别准备：**版头与风格遵循视觉 Skill，保留视觉基准；数据阶段建立数据插槽、投放与召回关系。内容、视觉与真实供给最终在同一份 page.html 中装配。
3. **构建页面并交付**：构建阶段把数据接到真实卡片生成 `page.html`，检查后保存版本，进入生码/审批/发布。
4. **主 Agent 贯穿链路**：确认、委派、验收、二次编辑都由它承接；业务阶段不等于七个长期 Agent，独立子任务才适合并行。

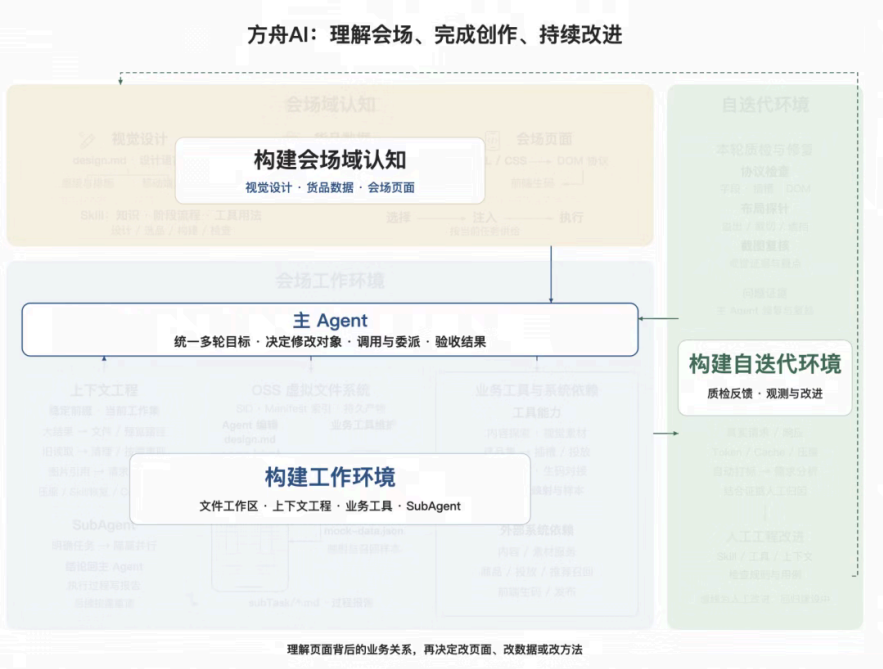
技术架构：构建方舟会场AI的 Harness 环境



Harness 的价值是把“会场怎么做”变成模型可理解、可操作、可检查的工作环境。常规的 Agent 架构图相信大家已经看腻了，可能翻来覆去就是那些 skill、tool、subAgent 之类的，基础的 agent 构建确实如此，而且成本已经很低了。

所以在方舟 AI 中，我们想着重表达的以下三类环境的构建：

1. **会场域认知环境**：视觉认知解释会场的设计语言，数据认知解释供给归属，生码认知理解数据协议；核心通过 Skill 渐进式加载。
2. **业务 Agent 的工作环境**：通用的 Agent 环境暂且不讲，在方舟 Agent 的执行过程中，如何建设工作区与操作环境？会场域认知知识怎么来？业务工具怎么来？如何通过 subAgent 降低耗时以及隔离上下文？上下文工程需要怎么优化？
3. **Agent 的自迭代环境**：我们希望设计阶段，在设计语言规范下能够最大程度的发挥视觉的创造力，在动态数据协议下最大程度的灵活调整货品，但是如何保障交付的确定性？以及如何监控观测运行过程，使其优化迭代我们的 Agent？



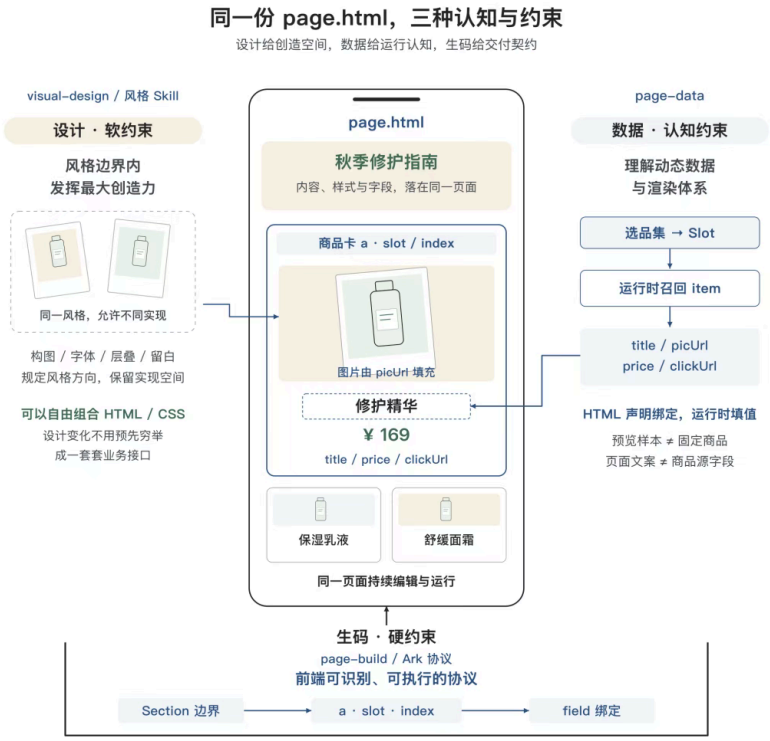
03 构建Agent的会场域认知

历如何做到内容策划以及淘内货品挖掘

会场生成的第一步，是根据用户诉求进行会场的策划以及货品供给的生成，这块有单独的文章进行展开，这里不再赘述了。

最重要的产物：承载丰富业务语义的html

- 一份 HTML，承载着三项约束

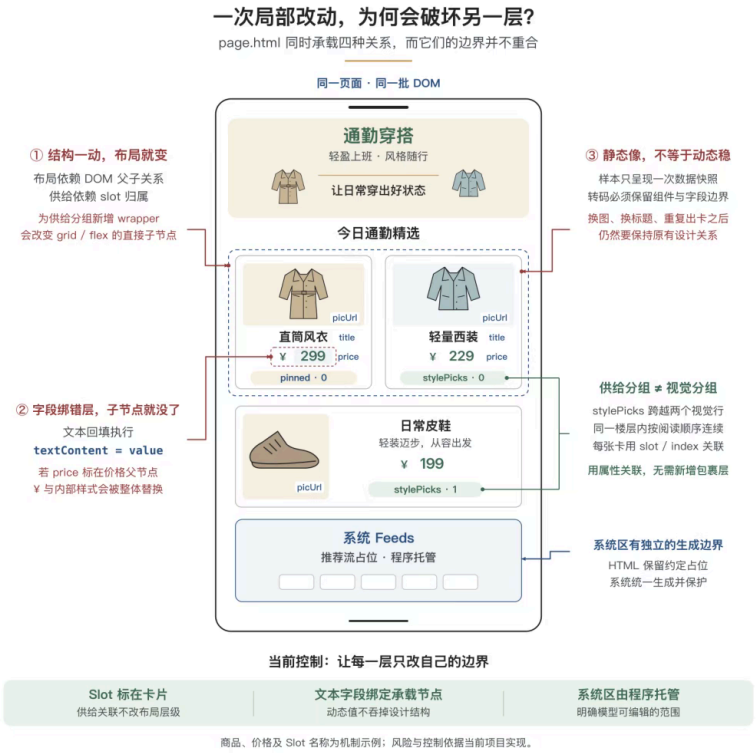


html只是底层的协议，让AI随便生成一张好看的html是比较容易的，但如果希望他可用、可运营、可投放，那还有很多事情和约束需要去做。

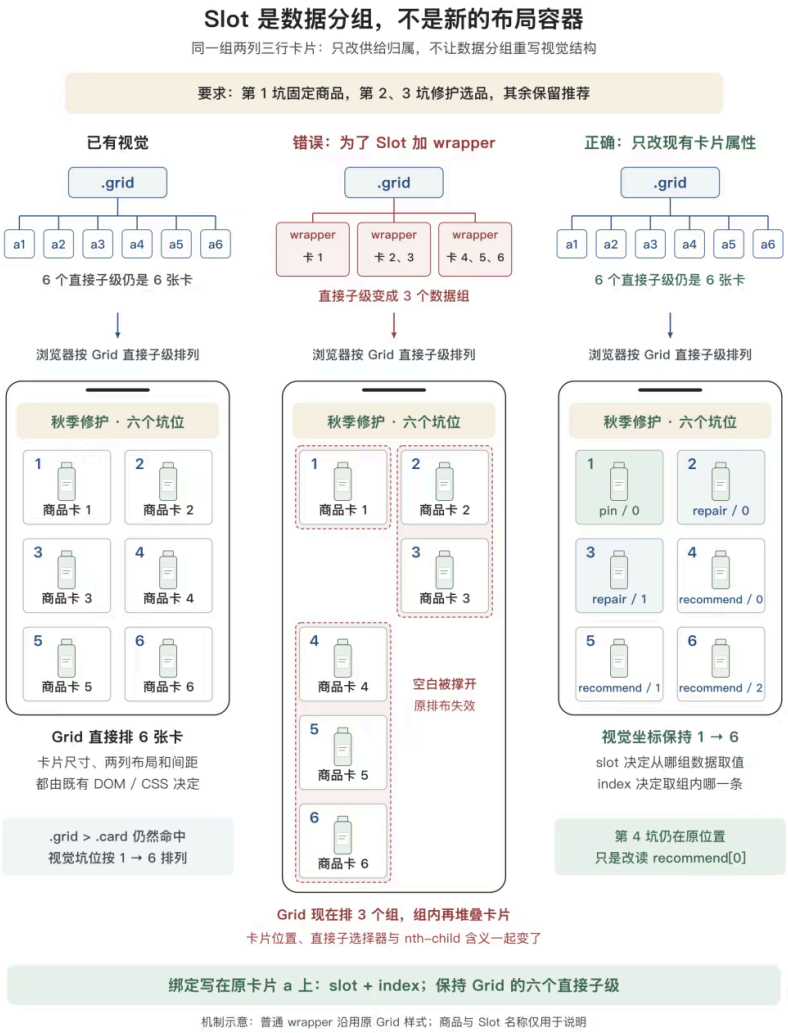
1. **视觉设计是软约束**：`visual-design` 与风格 Skill 守住气质、视觉层级和关键特征，相框、字体、层叠、留白仍可组合——让模型把设计做好，而不是照模板填空。
2. **货品数据是认知约束**：`page-data` 建立选品、Slot 与召回认知，字段协议规定内容如何进入页面；AI 要分清哪些内容随页面保留、哪些来自商品字段。
3. **生码是硬约束**：`page-build` 与 Ark 协议规定 Section、卡片、字段的表达与装配边界；外观自由，节点归属、字段位置、动态组件输入必须明确。

- page.html：AI设计的冲突中心

同一棵 DOM 同时承载布局、供给、字段绑定与转码要求。这些关系可以对应，却不能强行合并成同一层级；局部修改可能改变另一环节赖以成立的前提。

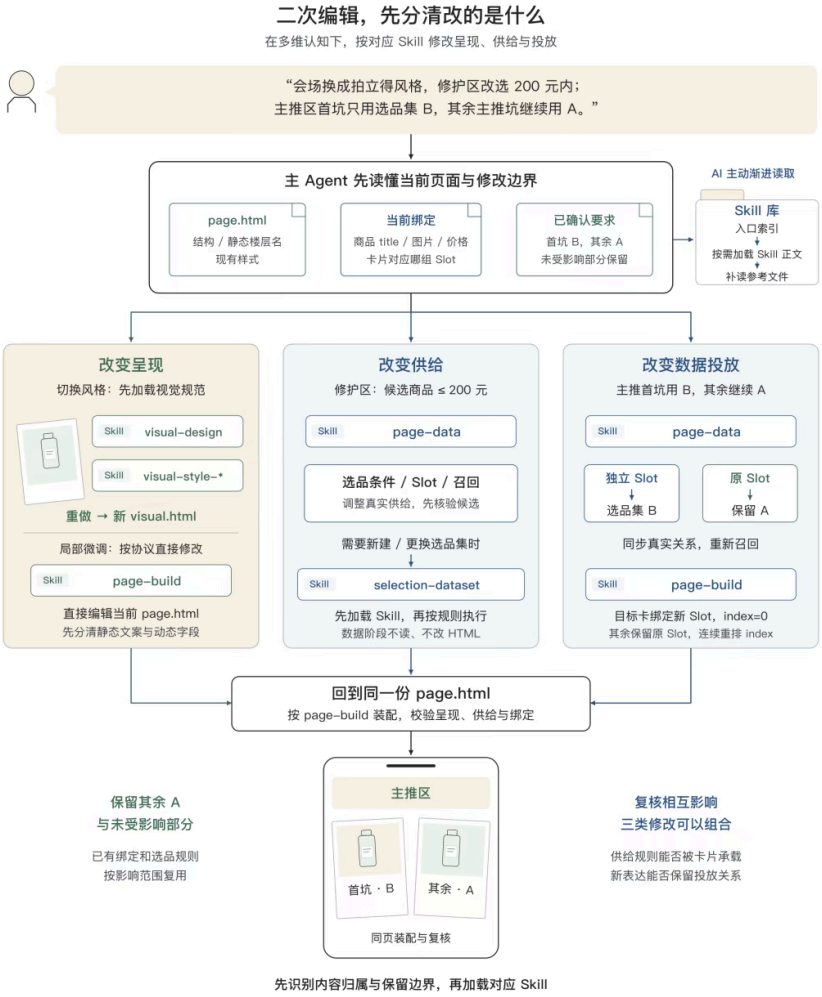


1. **同一棵 DOM 被三种方式解释**：视觉设计看dom结构与盒模型；数据链路识别 Slot、组内位置与字段承载节点；转码链路提取楼层、组件与样式关系。
2. **视觉分组 ≠ 供给分组**：同楼层可混排不同 Slot，同 Slot 卡片可跨行、异样式。
3. **加 wrapper 会改变布局前提**：为 Slot 包一层容器，Grid/Flex 直接子项就变了，跨行跨列与位置选择器受影响；`display: contents` 也会改变 DOM 父子关系。
4. **供给写进属性，布局保留层级**：协议把 `data-ark-slot`、`data-ark-index` 直接写在卡片 `<a>` 上，运行时按 `a[data-ark-slot][data-ark-index]` 定位卡片并绑定字段（`Render.ts`），两种分组分别表达。



- 在多维认知下进行二次编辑

同一张页面上的修改，可能改变呈现、货品供给，也可能改变某个坑位的数据投放。AI 先识别诉求与影响范围，再从 Skill 库的入口索引主动选择能力、按需加载正文，并沿规范补读参考文件。三个方向分别修改，混合诉求按依赖顺序处理，最终回到同一份页面校验。



运营对会场页面的修改，首先要按照不同对象进行分析，不同对象的编辑过程也不一样。

1. **改变视觉与样式，按视觉与页面 Skill 修改。** 整体换风格时加载 `visual-design` 和选中的 `visual-style-*`，先生成新视觉稿，再按 `page-build` 构建；字号、静态文案等局部微调，加载 `page-build` 后直接编辑当前 `page.html`。Skill 给出修改边界，未受影响的结构与绑定继续保留。
2. **改变供给，修改真正的选品与召回链路。** 例如“修护区换成 200 元以内的商品”，由 `page-data` 定位受影响 Slot；需要新建或替换选品集时，先加载 `selection-dataset` 校验候选、创建选品集，再同步供给与召回。数据就绪后，由 `page-build` 更新受影响卡片的绑定。
3. **改变数据投放，明确哪个坑使用哪套供给。** 例如“左侧主推坑只投选品集 B，其他坑继续用 A”。按 `page-data` 的固坑规则拆出独立数据插槽，原数据插槽继续服务其余卡片；再按 `page-build` 只更新值会场中。
4. **改文案，先分清静态内容与动态字段。** 楼层标题可以直接改；商品卡内的名称、价格、卖点要沿 `data-ark-field` 判断来源，动态值会由运行时替换。

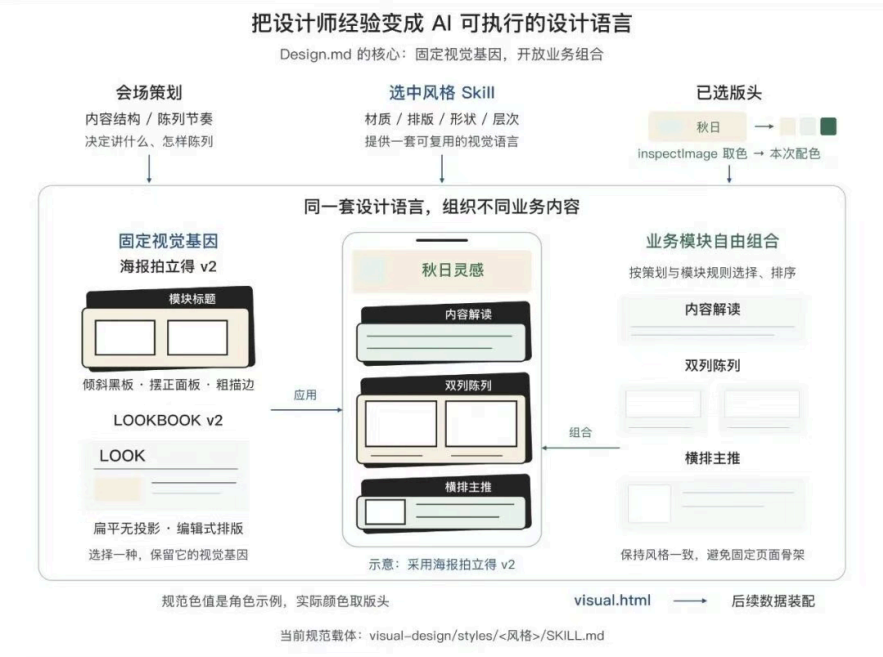
修改完成，需要复核三类认知是否仍然一致。 新供给要能被现有卡片承载，新呈现要保住字段替换和转码边界，新投放规则要落到真实 Slot 与目标卡片的映射。

如何保证视觉达到淘天设计水准？

- 视觉风格Skill：把设计师的经验变成 AI 的设计语言

我们和设计师合作，设计师针对淘内的内容型会场产出了 30+ Design Skill，把会场设计语言变成 AI 可理解的 skill，情绪、材质、排版、组件结构、色彩角色和禁用规则。并根据实际策划和货品进行自适应调整和创作。

例如“海报拍立得”保留倾斜黑板、摆正面板和粗描边；LOOKBOOK 强调扁平、无投影和编辑式排版。



- 保证视觉风格的效果与遵循度

风格规范要在长链路里持续生效，难点在于任务目标会变化：设计追求表现力，数据阶段关注供给，页面构建阶段关注协议。

我们先独立完成视觉创作、保存产物，再进入供给与页面构建；后续对话通过规则持久化、压缩恢复、编辑保护与质检反馈，持续提高视觉 Skill 的遵循度。



1. **先让当前任务专注视觉。** 用户确认版头和风格后，主 Agent 加载 `visual-design` 与 `visual-style-*`。这一阶段专注设计语言、布局和稳定盒模型，不同时处理 Slot、召回和最终绑定，减少多种规则争抢注意力；生成前先明确不可丢失的视觉基因。
2. **精确色值交给工具，视觉表达留给 AI。** 先使用工程手段进行像素采样和统计返回底缘色、主色簇等，给出稳定、可复核的色值，减少模型看图猜 hex 的偏差。AI 再按风格规范把它们映射到背景、主色和强调色
3. **把视觉成果保存成独立产物。**通过 `designPageVisual` 生成 `{主题}-{风格}-visual.html`，先确定布局、配色、字体与装饰，再进入数据和页面构建。
4. **数据装配遵守已有视觉结构。** `page-data` 准备供给与召回，`page-build` 进**行页面协议构建。**
5. **让规则随会话延续，压缩后自动恢复。** Skill 工具将加载的完整正文包装为带名称的 `<skill-loaded>` 消息，并独立持久化。
   a. 普通轮次沿用会话历史，微压缩不把这类规则消息当作普通 `read` 结果清理；
   b. 历史重建或完整压缩后，按加载记录和预算重新恢复正文，超长时保留截断与补读提示。
   c. 视觉规则因此有专门的保留与恢复路径，避免只剩一句风格摘要。

AI 如何在 HTML 上运营真实商品？

- 让 AI 建立会场数据的世界观

我们会场的商品数据并不是 html 中的一个 div，底层是一套数据运营和在线链路渲染的逻辑，需要让 AI 理解链路 才能正确修改。



1. **html的整体结构**：
    a. 通过 `page-build` 协议定位 Section、商品卡与字段节点；卡内标题/价格/图片是动态字段，楼层名可能是静态文案。
    b. `data-ark-index="0"` 只是组内位置，不代表固定商品；固定主推回到选品与投放规则，改字号留在展示层。
2. **会场数据的核心业务供给**：
    a. 通过skill让AI清楚楼层、商品区、数据插槽、选品集、商品 等实体模型之间的关系。
    b. 从 `data-ark-slot` 找到 `page-data.json` 映射，关联真实 Slot、投放与选品集； `selection-dataset` 说明 ID、类目、价格带如何变成候选。

- 一句货品运营诉求，AI 怎样推导出数据方案？

举一个例子，用户提问 “第一个坑位固定主推 A，紧接两位只放修护类，其余继续个性化，布局不动”

这个对话中，没提选品集与数据，但AI需要了解页面样式、商品坑排版、货品圈选范围、推荐方式与编辑修改的边界。



1. **定位现状**：“第一坑”是位置不是商品 ID，在楼层商品区中理解动态商品数据渲染逻辑。
2. **用 Skill 建立准绳**：`selection-dataset` 说清有效选品条件；`page-data` 说数据插槽的共享规则、独立商品区运营要独立投放单元。
3. **落实为数据方案**：A 用商品 ID 条件建独立选品集与 `pin` Slot；修护区建 `repair` Slot；其余保留原 `recommend`。
4. **先核验供给再执行**：候选不足要暴露，不能写静态卡片掩盖。
5. **落到卡片并验收**：主推位绑 `pin/0`，修护区绑 `repair/0、1`，其余继续引用原组；检查四个要求同时成立。

生成线上会场页面，进行C端投放

通过方舟Agent搭建的会场，可以和普通方舟会场一样，支持发布并投放手淘或端外，甚至也支持直接添加现有的天马模块。

会场前期设计产出的html只是一份承载了业务语义（视觉设计和货品数据）的协议HTML，实际生产环境的会场页面还有很多事情要做，比如性能要求、埋点、各种屏幕适配等等，这部分也有单独的文章展开。

04 构建Agent的会场工作环境

这部分是通用的AI Agent Harness建设的内容，内外网已经有很多最佳实践，这里就不过多赘述了，只选取几点实际落地比较有印象的点展开讨论。

实际业务中理解上下文与cache

- 做好上下文管理的前提：进行重要性分层



我们根据我们实际的上下文进行了一轮分析，根据其重要性排序

- **P0：核心约束和执行事实，**比如系统提示词、用户目标等
- **P1：已加载的关键Skill**，比如设计风格、页面协议
- **P2：当前判断所需的信息结果**，比如当前html、当前截图
- **P3：已过时、可重取的信息结果**，比如read工具结果、早期截图、会场信息等

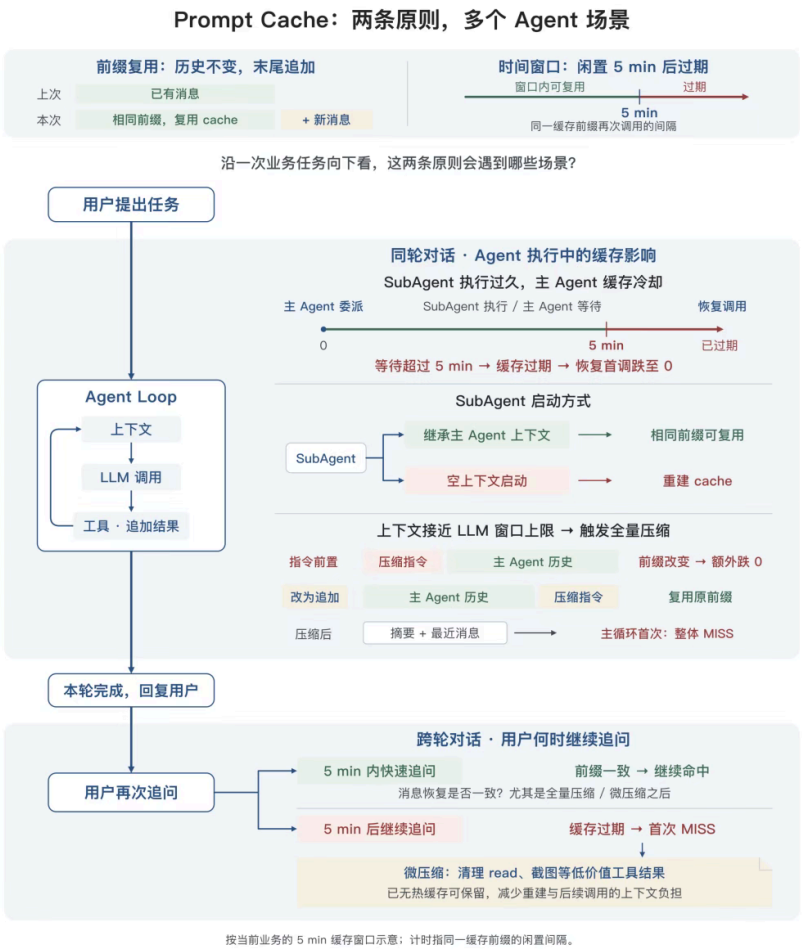
对于重要性很高的上下文 比如Skill 要注意在全量压缩的时候进行显示的恢复。

对于重要性很低的上下文（比如read工具结果、截图工具结果等）在合适的时机将其清除，合适的时机 比如 缓存冷却的时候。

- prompt cache 在实际业务中并不简单

在没有实际做agent业务项目之前，我对prompt cache的理解基本只在两点：**保持前缀一致、默认缓存5min。**

但实际上这两点，在agent loop、多agent架构的场景下 其实衍生出了很多种情况，每种情况的场景和解决优化的思路也各不相同。



首先可以把上下文分为同轮和跨轮：

- **同轮对话：**
  - 基本原则：消息的添加仅新增到末尾，过程中不要篡改历史消息。
  - 上下文全量压缩：如果遇到上下文过长引起的全量压缩会出现一次整体 miss。
  - SubAgent：
    - subAgent 的类型有继承上下文和空上下文启动，空上下文启动会重建cache。
    - 如果 subAgent 执行时间过长，超过 5min，主 agent 也会发生一次 miss。
- **跨轮对话：**

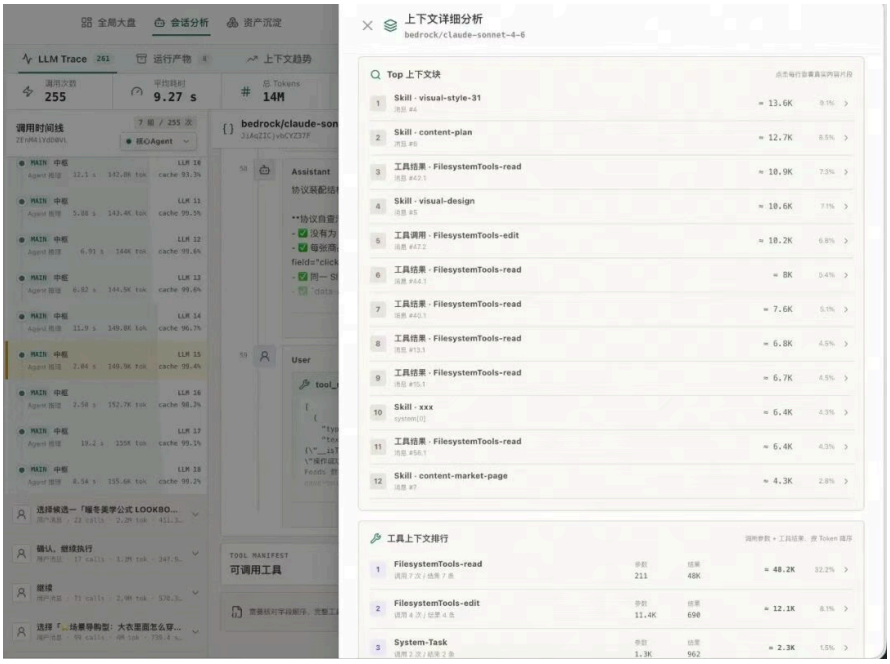
- 5min 内快速追问：这个时候重要的是消息恢复机制，尤其对于上下文压缩、微压缩等场景下消息恢复要保持一致。
- 5min 后继续追问：用户轮次的不可控，但由于缓存一定 miss，所以可以做一些优化，比如 **微压缩** 将一些不重要的工具结果删除 比如 read 工具。

所以其实如果你的业务系统整体 prompt cache 命中率不高，可能和你的上下文工程无关 一些其他操作会降低命中率，比如对话周期很长，用户经常隔很久继续对话；比如用的模型上下文窗口只有 200k，经常触发全量压缩等。这些对 cache 命中造成的具体影响还需要进一步下钻分析。

- 上下文与 cache：成本和效果的权衡

上下文工程和 cache 命中率其实是一种权衡，我们既不能完全忽略 prompt cache（真的很省钱），但也不能一味的追求 cache 命中率，上下文中塞入过多无用的信息。当缓存冷却时（超过 5min），将一些不重要的信息清理 是一个很好的方案。

比如从上下文的埋点分析来看，read 工具占比还是非常高的。

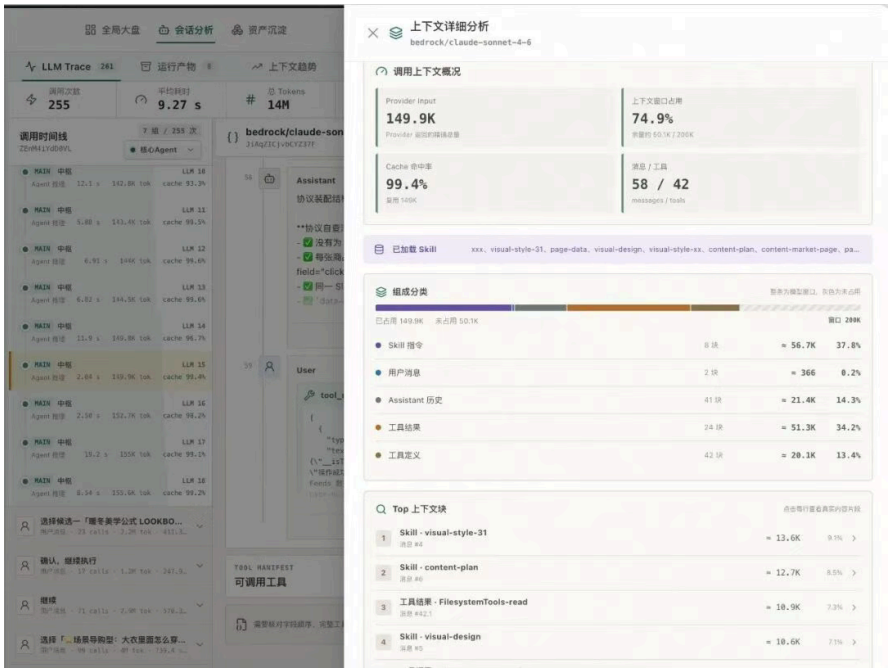


- 监控上下文 trace，从模型视角观测发现问题

在实践过程中，记录 llm trace 是极其重要的。我们做了那么多上下文的处理，如果连每次请求给 llm 的输入都不清楚，那就白做了。

在我们的观测工作台，我们详细记录了每次 llm request 的输入输出，也辅助我们看到 llm 如何决策。

同时，我们也详细记录了每次 trace 的上下文组成，一些占用比较大的上下文块、工具输出等，辅助针对上下文进行下钻分析。

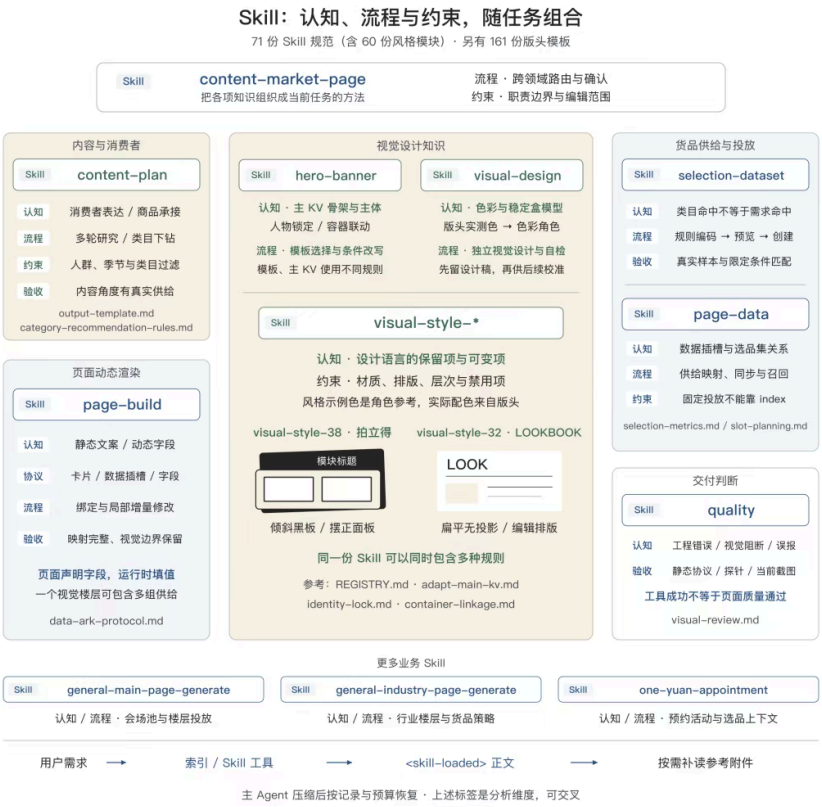


通过 Skill 供给会场知识与流程

为了支撑上述 AI 的会场域认知，我们有大量的 skill 体系，这些 skill 相互配合，提供了大量会场知识认知，也提供了流程、协议、约束、产物等。一个好的 skill 需要说清楚表达的认知，也要有明确约束和验收条件。

内容为谁服务、商品从哪里来、设计保留什么、线上怎样渲染，以及结果凭什么过关。

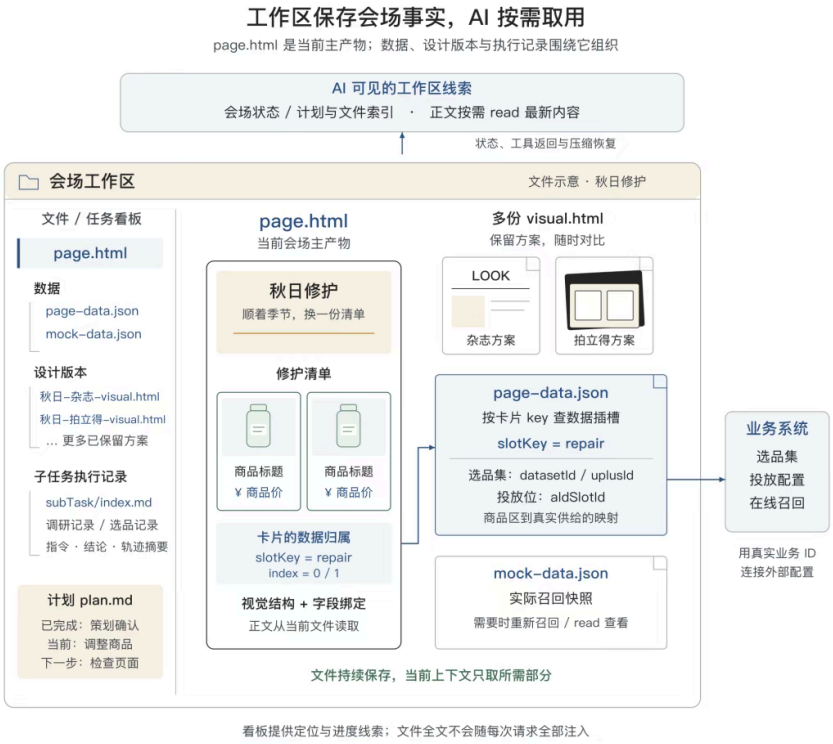
下面按知识领域展开；一个请求可以同时调用多个领域，同一份 Skill 也可以兼具认知、流程、协议和验收规则。



- **内容认知：**把主题落到消费者问题与商品承接。
- **视觉认知：**知道核心的设计语言，以及什么该保留、什么可以变。
- **供给认知：**区分需求、选品规则和实际投放。
- **渲染认知：**静态文案如何渲染，动态商品数据如何渲染
- **验收认知：**判断问题是什么，以及应该回哪里改。

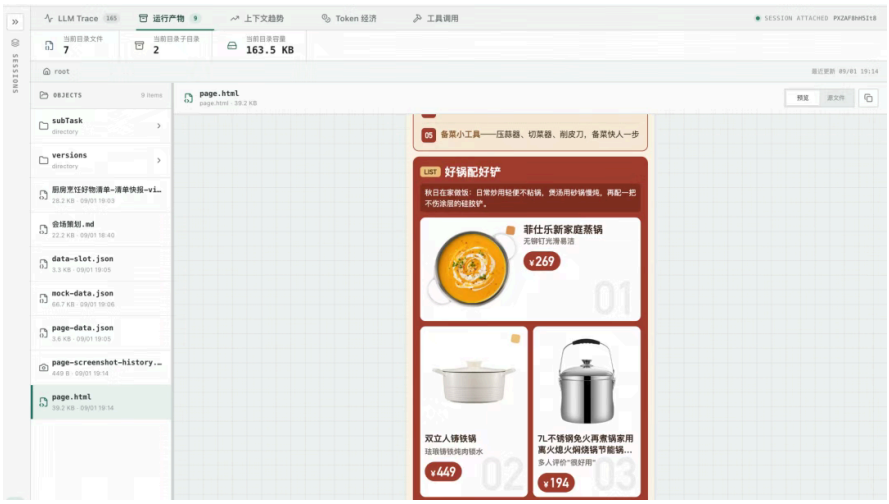
为云端业务 Agent 设计的工作区

会场 Agent 的工作环境，要承接业务认知、页面产物和业务系统工具。我们把它拆成两部分：持续保存工作材料的 OSS 虚拟文件系统，以及按需调用的执行能力。工作区不仅存文件，还把复杂业务收敛成 AI 能阅读、能交接的任务材料。



1. **文件语义保留，持久状态放到 OSS。** AI 通过 `ls/read/write/copy/edit` 操作工作材料，底层按 SID 映射到 OSS 路径。写入先落 OSS，再登记产物清单。会话释放不删除这些产物，后续执行可以继续读取，工作进度不依附于某个运行进程。
2. **通过文件收敛认知范围，只保留必要业务语义。** AI 需要理解“修护区用哪套选品、主推卡绑定哪个数据插槽”。`page-data.json` 提供 `slotKey / aldSlotId / datasetId / cardCode` 等映射；底层投放-选品的复杂模型关系由业务工具处理。AI 仍要懂投放逻辑，但不用在每轮对话里重建整套后端对象关系。

同时，我们也可以在观测工作台观测工作区的产物。



多Agent系统设计：SubAgent的收益与成本

- SubAgent的思考与优劣

SubAgent 的收益很直接：把大段探索放进独立上下文，让专业规则更聚焦，让互不依赖的任务并行。但会场业务会连续修改同一份页面，一轮任务做完之后，还要让下一轮对话沿着同一套用户确认和业务事实继续。先把几个容易混在一起的维度分开。

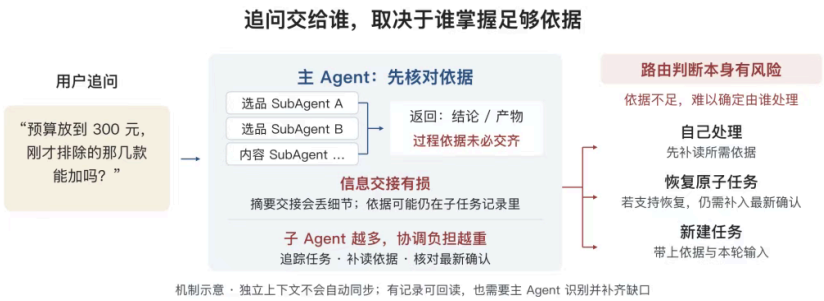
维度	可选形态	真正改变的是什么
初始上下文	干净上下文	从任务指令、必要材料和规则起步，减少无关历史；背景不够时补齐，专业能力由模型、Skill 与工具供给。
	继承 / fork	继承启动时的父 Agent 消息快照，之后各自演进。历史较全，但上下文更大；继承不代表后续实时同步。
专长配置	通用 / 专用	通过提示词、Skill、工具和权限配置专业能力与执行范围；可与不同的上下文、状态策略组合。
状态生命周期	调用级 / 任务型	每次围绕一次委派启动，结束后可保留结论、产物和记录；后续不默认续接原执行状态。
	跨调用持久	保存可恢复的会话状态与身份，后续可继续；不等于进程常驻，也不会自动获得主 Agent 后来的用户确认。

三个维度可以组合：专用选品 Agent 也可以每次干净启动；任务型子 Agent 也可以继承上下文。

收益明确，协作成本也要算进去。 独立探索能隔离上下文、缩短并行任务的等待时间；任务拆解、信息交接、重复推理和汇总验收也会增加开销。多轮会场编辑尤其容易遇到三个问题：

1. 交回了结果，没交齐依据。 选品子 Agent 只回“已创建选品集”，主 Agent 知道用了哪个 ID，却未必知道哪些商品因价格排除，哪些因敏感肌限制排除。用户放宽预算后，前者可以重查，后者仍要保留。只有结论，没有条件与可回读证据，下一轮就容易把已经做过的判断再猜一遍。

2. 追问交给谁，要先知道谁掌握足够依据。 “预算放到 300 元，刚才被排除的那几款能加吗？”主 Agent 得先定位原任务、找到排除依据，再决定自己处理、恢复之前的子 Agent，还是补齐输入后发起新任务。之前的子 Agent 即使保留历史，也可能缺少最新确认；“选品专家”这个名字，不能替系统完成交接。



机制示意 · 独立上下文不会自动同步；有记录可回读，也需要主 Agent 识别并补齐缺口

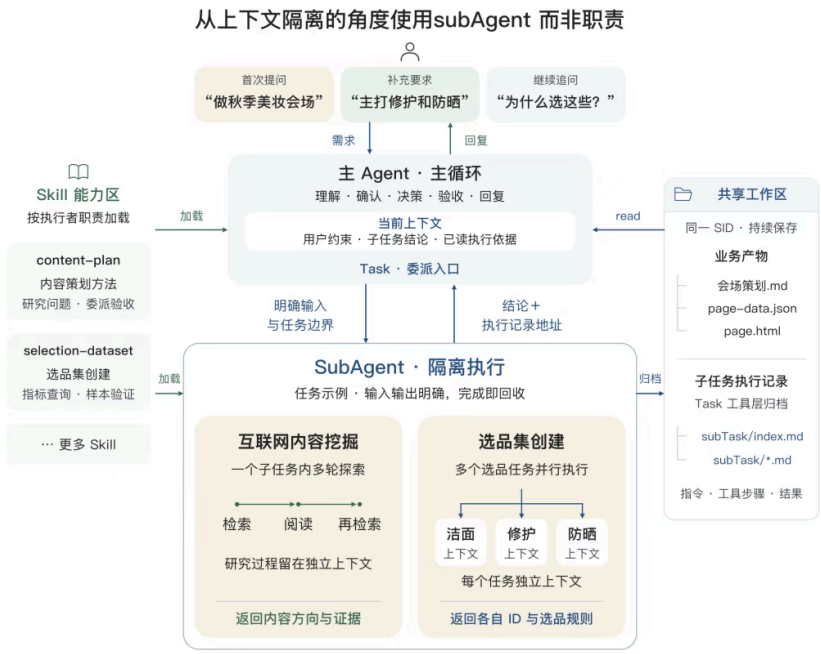
3. 上下文各自独立，修改仍汇入同一份会场。 设计侧按新确认改了卡片结构，绑定侧还基于旧节点形成修改，两份局部结果合在一起，就可能错绑或覆盖。继承上下文、保留会话、共享工作区，都不能代替当前版本核对、明确的写入范围与统一验收。把全部聊天记录互相转发，也不会自动解决这些问题。

再看“按什么边界拆 Agent”，可以比较三个视角。它们可以组合，并不是互斥、完备的三种架构。

1. **按职责拆：集中专业能力。** 比如策划、设计、前端各自配置规则和工具。边界清楚时便于维护；一次“换风格但保留这批商品”的需求却横跨多个职责，仍需有人掌握完整修改范围。角色配置也不意味着必须长期保留一个执行实例。
2. **按流程拆：分开决策与执行。** 比如 Planner 规划，Executor 执行。执行中发现供给不足，就要把新事实反馈给规划侧，多轮新诉求也可能使旧计划失效。如果只是同一个 Agent 内切换阶段，则是流程划分，不能仅凭两个名称就称为多 Agent。
3. **按任务拆：把委派边界落在一次可验收的工作上。** 主 Agent 保持业务主循环，把输入、输出和边界明确的任务交出去；专业能力仍可由 Skill 与工具供给。这种方式便于隔离上下文和安排独立任务并行，但拆解、验收和后续补读仍由主 Agent 承担。

- 从上下文隔离的角度使用SubAgent，子能力由Skill供给

我们的选择是：**主 Agent 持续负责用户对话和业务主循环，Skill 提供做事的方法，Task 工具提供委派与并行执行能力。**子 Agent 围绕一次明确任务工作，结束后交回结果。



1. 主原则：主 Agent 负责连续决策，Skill 与 Task 提供能力。

a. 用户提需求、确认方向、继续追问，都回到主 Agent，按职责加载 Skill，例如内容策划使用 `content-plan`，创建选品集的执行者加载 `selection-dataset`。需要委派时，再通过 Task 启动子任务。

2. **两个场景：**

a. **场景一：输入输出说得清，才把任务交出去。**

互联网内容挖掘可以交代主题、人群与研究问题，要求返回内容角度和证据；选品集创建可以交代类目、价格带和筛选要求，要求返回 ID 与实际规则。通过subAgent来进行上下文隔离。

b. **场景二：独立任务通过并行加速**

策划结构明确后，洁面、修护、防晒可以分别创建选品集，各自在局部上下文里查询指标、预览样本，再并行交回结果。通过独立任务并行缩短时间。

3. **如何应对信息损失：交接用“结论＋执行记录地址”，后续对话沿文件继续。**

a. 子任务结束后，执行记录记到工作区，Task 返回结论，并附上已归档执行记录的路径；结论进入主循环，过程保留为可重读的依据。

b. 用户再问“这批修护商品为什么被选进来，价格再降一点”，主 Agent 可以读取记录、核对当前产物，再决定自己调整或发起新任务。结论进入主循环，过程保留为可重读的依据。



质检：生成需要创造力，交付需要确定性

- 为什么重要？质检工具的三层设计

我们希望 AI 在设计阶段足够的灵活，包含了视觉效果的最大化、数据编辑的灵活性，但是在交付角度，质量必须过关。所以对质检工具提出了一个前提 两个基本要求：

- **一个前提**：不能阻碍AI在视觉设计、商品运营上的创造力
- **两个基本要求**：

  确定性高：这样保证交付的确定

  检测速度快：耗时短才能成为基础工具，随处编辑后随处调用验证。

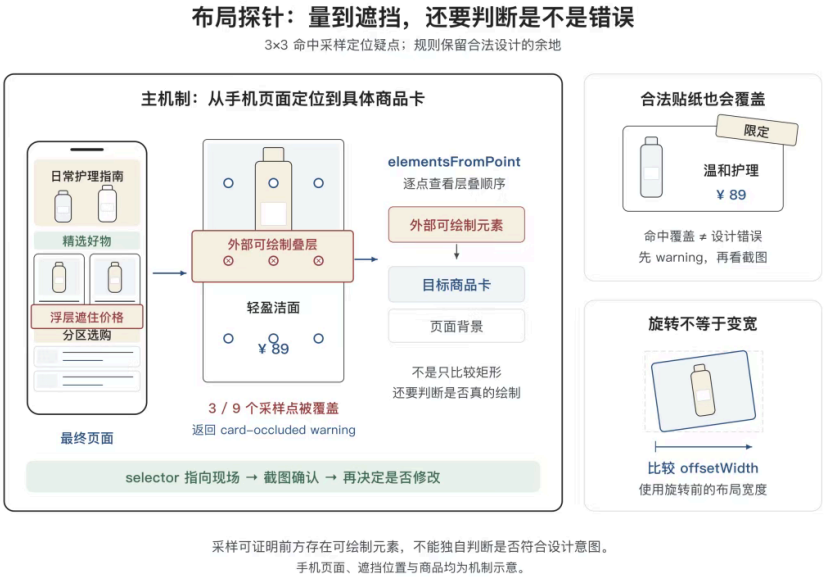
若为减少错误把生成限制成几套模板，设计的创造里也会一并消失。我们希望模型尽量发挥 HTML/CSS 表达力，把可验证的不确定性收进质检反馈。



检查层	看什么	给 Agent 什么反馈
静态协议	HTML 结构，Slot、index、字段，文件与数据覆盖	规则 ID、节点位置、修复建议
浏览器布局探针	渲染后的尺寸、裁切、溢出、重叠、遮挡、异常空白	实际几何与命中结果
截图复核	构图、风格、可读性，以及 warning 是否属于合理设计	当前页面的视觉证据

- 用确定性检测来判断不确定的难点

质检工具的实现并不如想象中的那么简单，本质上是 视觉设计的泛化性 与 确定性质量交付 二者之间的冲突。如果一味追求检测，追求确定性，会让 AI 失去创造力，下面举一个例子。



质检在实际的实践中，琐碎的问题比较多：

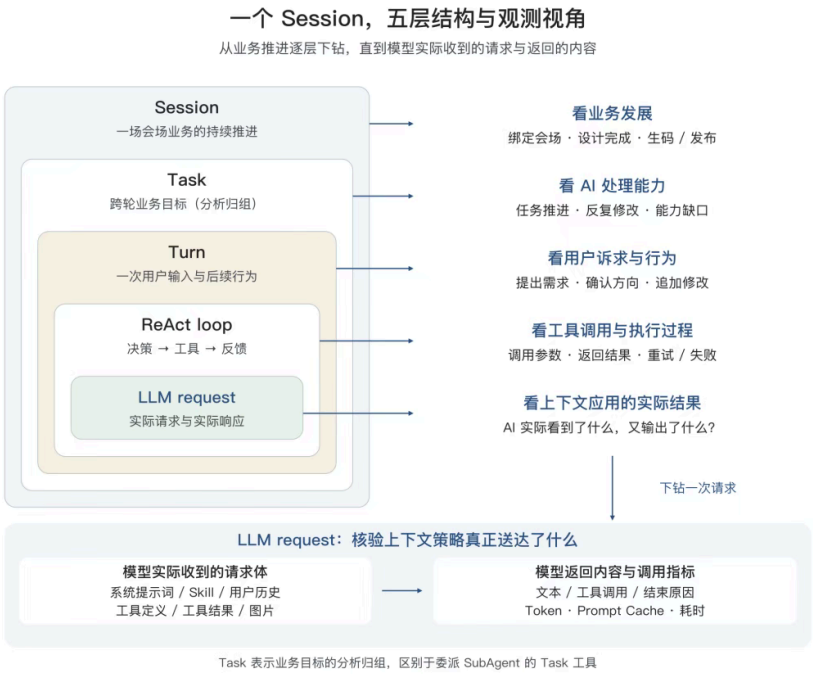
1. 先判布局条件：掉坑检查先确认共同父容器，区分显式缺格与末行不满；卡宽一致性要求等宽轨道、无跨列。
2. 旋转看正确尺寸：倾斜卡片用变换前 `offsetWidth` 判断原始卡宽，避免把拍立得倾斜当不齐。
3. 遮挡看实际绘制：3×3 采样 + `elementsFromPoint` 检查层叠；外部浮层可能是贴纸，作 warning 交截图复核。

允许设计 ≠ 豁免重叠：商品卡超阈值重叠仍报 error；Bento、旋转、横向滚动等合法反例需持续回归，**否则模型会被错误规则逼成同一版式。**

观测：解构会话，实现迭代闭环

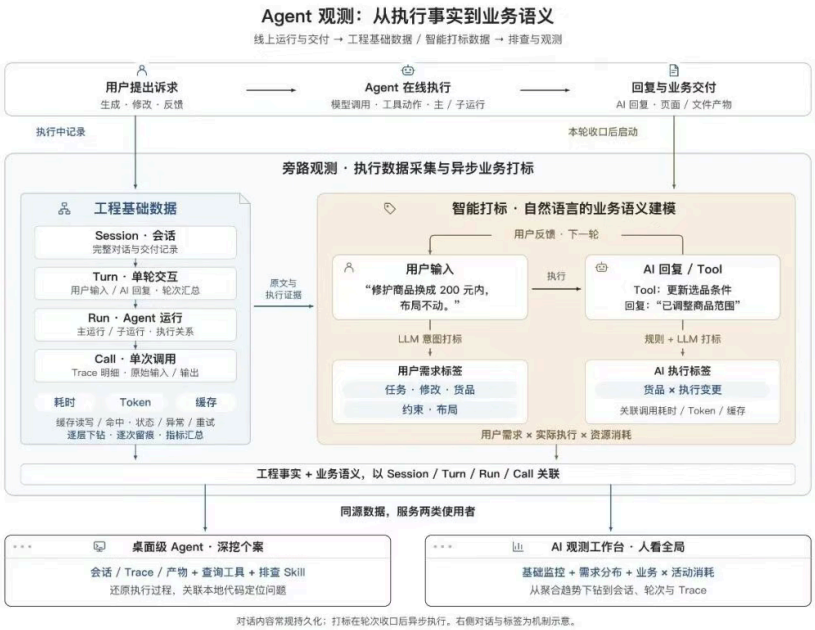
- 解构一个session的五层观测视角

状态里的消息还会经过 Skill、工具可见性、多模态与缓存处理。Provider 出口记录 SDK 最终发送的请求和实际收到的响应，才能确认规则是否加载、图片是否进入请求、字段是否可见、工具结果以什么形式进入模型上下文。



观察可以从 Session 下钻到用户 Turn，再看主/子 Agent 的请求与工具步骤：Turn 看用户诉求与执行结果，请求看模型实际输入输出，工具步骤看业务动作。Task 委派通过父子运行标识关联，ReAct 循环由请求和工具序列还原。

- 建立业务打标体系，对用户需求进行分析



我们对用户的对话进行llm打标，并不断更新打标服务，从用户对话中分析数据。

- **Agent的会场域认知**：目前这块还只是跑通当前业务的核心知识，在已有的需求中，有越来越多的业务接入，不同业务的数据投放诉求不同、不同业务的盘货诉求也不同，这些也需要一定的二三方开放能力建设。
- **Agent的工作环境**：在实际的会场设计业务场景中，对视觉、对货品的调整非常多，有几十轮甚至上百轮，在如此高频的长程任务中，如何做到上下文不丢失，保持AI的视觉创造力、货品运营灵活度，是需要持续探索的。
- **Agent的自迭代环境**：目前已经初步建立起AI观测工作台，但依然缺少Task维度的分析、Turn维度的深度分析，以及更加自动化的反哺到核心Agent的迭代中，仍需要不停设计。



本文作者巷年，来自淘天集团–营销&交易技术团队。该团队负责淘天集团核心电商的营销业务，包括但不限于：超大型营销活动导购、优惠/红包/互动玩法等，参与或主导营销业务的AI应用，构建AI驱动的营销技术工程体系，运用AI思维与工程手段，主动探索创新性解决方案，为业务带来突破性变化。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
966篇原创内容
公众号

阅读 2295

留言

写留言

还没有留言，来写首评